PBL · Segurança e Criptografia · Projeto SecureDocs (TechSecure)

Missão 3 – Milhares de documentos foram roubados

Resumo: criptografia simétrica (DES, RC5 e AES), escolha do algoritmo e o problema da distribuição de chaves

Equipe de Especialistas em Segurança e Criptografia: Aluno 1 · Aluno 2 · Aluno 3 · Aluno 4 · Aluno 5 | Outubro de 2026

1. Contexto da missão
2. Fundamentos de criptografia simétrica
3. Implementação do DES
4. Implementação do RC5
5. Implementação do AES
6. Modos de operação e formato de arquivo
7. Experimento: Arquivo → AES → Cifrado → AES → Arquivo
8. Testes e ataques realizados
9. Comparativo DES × RC5 × AES
10. Justificativa: qual algoritmo usar no SecureDocs
11. Nova descoberta: como entregar a chave secreta?
12. A biblioteca entregue
13. Referências

1. Contexto da missão

Um invasor copiou arquivos do servidor do SecureDocs. Eles estavam protegidos por **cifras clássicas** (substituição e transposição, como César e Vigenère). Essas cifras mantêm a estatística da língua: a frequência das letras e os padrões repetidos aparecem no texto cifrado, e a análise de frequência ou a força bruta sobre um espaço de chaves pequeno revela o conteúdo em minutos. A pergunta da TechSecure é:

> *"Mesmo que um invasor consiga copiar nossos arquivos, como podemos impedir que ele conheça seus conteúdos?"*

A resposta é cifrar os documentos em repouso com uma **cifra simétrica moderna**. Ela deve resistir à criptoanálise e ter uma chave grande o bastante para tornar a força bruta inviável hoje e no futuro. Para isso a equipe estudou, implementou e comparou o DES, o RC5 e o AES.

2. Fundamentos de criptografia simétrica

Na criptografia **simétrica** a mesma chave secreta *K* cifra e decifra: $\mathrm{C} = \mathrm{E}_{\mathrm{K}}(\mathrm{P}) \quad \mathrm{P} = \mathrm{D}_{\mathrm{K}}(\mathrm{C})$

- **Cifra de bloco:** transforma blocos de tamanho fixo (64 bits no DES e no RC5-32, 128 bits no AES). Para mensagens maiores é usado um *modo de operação* (seção 6).
- **Confusão e difusão (Shannon, 1949):** a confusão esconde a relação entre chave e cifrado (S-boxes, rotações dependentes de dados). A difusão espalha a influência de cada bit do texto por todo o bloco (permutações, MixColumns). Na prática, mudar um bit da entrada muda cerca de metade dos bits da saída (*efeito avalanche*).

- **Estruturas:** *rede de Feistel* (DES, e de forma generalizada o RC5): cada rodada processa metade do bloco, e a decifragem usa o mesmo circuito com as subchaves invertidas. *Rede de substituição-permutação* (AES): todas as rodadas transformam o bloco inteiro, e cada etapa tem uma inversa própria.
- **Princípio de Kerckhoffs:** o algoritmo é público; a segurança depende apenas do sigilo da chave. Por isso o tamanho da chave e a resistência à criptoanálise são os critérios centrais.

3. Implementação do DES

O **Data Encryption Standard** (IBM/NSA, FIPS 46, 1977) cifra blocos de **64 bits** com chave de **64 bits, dos quais 56 são efetivos** (8 são de paridade), em **16 rodadas de Feistel**. Arquivo: `securedocs_crypto/des.py`.

3.1 Algoritmo implementado

1. **Permutação inicial (IP):** reordena os 64 bits do bloco segundo a tabela `IP`, que divide o resultado em duas metades de 32 bits, L_0 e R_0.
2. **16 rodadas de Feistel:** $L_i = R_{i-1} \quad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$
3. **Função f (`_funcao_f`):**
   - *Expansão E:* 32 → 48 bits (bits das bordas de cada grupo são duplicados);
   - *XOR* com a subchave K_i de 48 bits;
   - *8 S-boxes*: cada grupo de 6 bits vira 4 bits. O 1º e o 6º bits escolhem a linha (0–3) e os 4 bits centrais escolhem a coluna (0–15). É a **única etapa não linear** do DES;
   - *Permutação P* sobre os 32 bits resultantes.
4. **Troca final e IP^{-1}:** após a 16ª rodada as metades não são trocadas ($R_{16}\|L_{16}$), e aplica-se a permutação final `FP`.
5. **Escalonamento de chave (`gerar_subchaves`):** `PC-1` descarta a paridade (64 → 56 bits) e separa as metades C e D de 28 bits. Em cada rodada, C e D giram 1 ou 2 posições à esquerda (`SHIFTS`), e `PC-2` escolhe 48 bits para formar K_i.
6. **Decifragem:** pela propriedade da rede de Feistel, é a mesma função com as subchaves em ordem inversa (`self._subchaves[::-1]`).

Decisões de implementação: o bloco é manipulado como um inteiro Python. Uma única função genérica `_permutar(valor, tabela, bits)` aplica todas as tabelas do padrão (IP, FP, E, P, PC-1, PC-2), que foram copiadas literalmente do FIPS 46-3 (posições de 1 a 64, bit 1 = mais significativo). Também foi implementado o **3DES-EDE** (`TripleDES`): $C = E_{K3}(D_{K2}(E_{K1}(P)))$, com chave de 16 ou 24 bytes.

3.2 Segurança

> **Inseguro.** O espaço de 2^{56} chaves foi quebrado por força bruta em 1998 pela máquina *Deep Crack* da EFF (56 horas) e em 1999 em 22 horas, com a ajuda do distributed.net. Também existem a criptoanálise diferencial (2^{47} textos escolhidos) e a linear (2^{43} textos conhecidos). O bloco de 64 bits permite ataques de aniversário (*Sweet32*) após ~32 GB cifrados com a mesma chave. O 3DES amplia a chave (112 bits efetivos), mas herda o bloco de 64 bits, é 3× mais lento e foi proibido pelo NIST para cifrar novos dados a partir de 2024 (SP 800-131A Rev. 2).

4. Implementação do RC5

O **RC5** (Ron Rivest, 1994; RFC 2040) é uma cifra de bloco **totalmente parametrizada**, notada RC5-*w*/*r*/*b*: palavra de *w* bits (16, 32 ou 64, logo bloco de 2*w* bits), *r* rodadas (0–255) e chave de *b* bytes (0–255). A configuração de referência é a **RC5-32/12/16**: bloco de 64 bits, 12 rodadas e chave de 128 bits. Arquivo: `securedocs_crypto/rc5.py`.

4.1 Algoritmo implementado

Usa apenas três operações, todas módulo 2^w: soma (+), XOR ($\oplus$) e **rotação dependente dos dados** ($\lll$). O número de posições rotacionadas vem do próprio texto, o que dá a não linearidade sem S-boxes.

1. **Expansão de chave (`_expandir_chave`):**
   - a chave é convertida em *c* palavras L[ ] (little-endian);
   - a tabela S de $t = 2r+2$ palavras é inicializada com as "constantes mágicas" $P_w = \text{Odd}((e-2)\cdot 2^w)$ e $Q_w = \text{Odd}((\varphi-1)\cdot 2^w)$: S[0] = P, S[i] = S[i−1] + Q;
   - a chave é misturada em S com $3\cdot\max(t, c)$ iterações: $A = S[i] = (S[i]+A+B) \lll 3$; $B = L[j] = (L[j]+A+B) \lll (A+B)$.
2. **Cifragem (`encrypt_block`):** A = A + S[0]; B = B + S[1]; para i = 1..r: $A = ((A \oplus B) \lll B) + S[2i]$ $B = ((B \oplus A) \lll A) + S[2i+1]$
3. **Decifragem (`decrypt_block`):** percorre as rodadas de trás para frente, com subtração e rotação à direita: $B = ((B - S[2i+1]) \ggg A) \oplus A$; $A = ((A - S[2i]) \ggg B) \oplus B$; e por fim B −= S[1], A −= S[0].

Decisões de implementação: a classe `RC5(chave, rodadas=12, tamanho_palavra=32)` aceita qualquer $w \in \{16, 32, 64\}$. A máscara `(1 << w) - 1` faz as reduções módulo 2^w, e a rotação é feita com `n % w`, porque só os $\log_2(w)$ bits menos significativos do valor importam.

4.2 Segurança

> **Não recomendado.** O RC5-32/12 é vulnerável à criptoanálise diferencial com cerca de 2^{44} textos escolhidos (Biryukov & Kushilevitz, 1998), por isso recomenda-se pelo menos 18–20 rodadas. O bloco de 64 bits tem o mesmo limite de aniversário do DES. O RC5 foi patenteado até 2015, nunca foi padronizado pelo NIST e não tem aceleração em hardware. Uma chave RC5 de 64 bits foi quebrada pelo distributed.net em 2002. É uma cifra elegante e rápida em software, mas fica atrás do AES em análise pública, padronização e suporte.

5. Implementação do AES

O **AES** (algoritmo Rijndael, de Daemen e Rijmen) venceu o concurso público do NIST e virou o **FIPS 197** em 2001. Ele cifra blocos de **128 bits** com chaves de **128, 192 ou 256 bits**, em 10, 12 ou 14 rodadas de uma **rede de substituição-permutação**. Arquivo: `securedocs_crypto/aes.py`.

5.1 Algoritmo implementado

O bloco é tratado como um **estado**: uma matriz 4×4 de bytes preenchida coluna a coluna. Os bytes são elementos do corpo finito **GF(2^8)** com o polinômio m(x) = $x^8+x^4+x^3+x+1$ (0x11B). A soma é XOR e a multiplicação é feita por `xtime` / `gmul`.

Etapa	O que faz	Propriedade
SubBytes	Substitui cada byte pela S-box: inverso multiplicativo em GF(2^8) seguido de uma transformação afim ($\oplus$ 0x63).	Confusão (não linear)
ShiftRows	A linha *r* do estado gira *r* posições à esquerda.	Difusão entre colunas
MixColumns	Cada coluna é multiplicada pela matriz circulante [02 03 01 01] em GF(2^8).	Difusão dentro da coluna
AddRoundKey	XOR do estado com a chave da rodada (128 bits).	Introduz a chave

- **Fluxo:** AddRoundKey inicial; Nr−1 rodadas completas; a última rodada sem MixColumns.
- **Expansão de chave (`expandir_chave`):** gera 4·(Nr+1) palavras de 32 bits. A cada Nk palavras aplica RotWord, SubWord e XOR com a constante de rodada Rcon (potências de x em GF(2^8)). No AES-256 há um SubWord extra no meio de cada grupo.
- **Decifragem:** aplica as inversas (InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns com a matriz [0E 0B 0D 09]) e as chaves de rodada em ordem reversa.

Decisões de implementação: a **S-box não foi copiada de uma tabela**. Ela é *calculada* na importação do módulo (inverso por exponenciação a^{254} + transformação afim), e o teste confere os valores conhecidos (S[00]=63, S[53]=ED). As multiplicações por 2, 3, 9, 11, 13 e 14 são pré-calculadas em tabelas para acelerar MixColumns. O estado é uma lista de 16 inteiros em ordem de coluna (`estado[linha + 4·coluna]`).

5.2 Segurança

> **Seguro.** Após 25 anos de análise pública, o melhor ataque ao AES completo (biclique, 2011) reduz o AES-128 de 2^{128} para $2^{126,1}$ operações, uma diferença sem efeito prático. Contra computadores quânticos, o algoritmo de Grover reduziria a segurança à metade dos bits: o AES-128 cairia para 2^{64}, mas o **AES-256 manteria 2^{128}**. Por isso o AES-256 é a recomendação para dados que precisam continuar sigilosos por décadas (por exemplo, a suíte CNSA 2.0 da NSA).

6. Modos de operação e formato de arquivo

A cifra de bloco sozinha só processa 16 bytes. O módulo `modos.py` implementa, de forma genérica para qualquer cifra da biblioteca:

- **ECB:** cada bloco é cifrado isoladamente. Blocos iguais geram cifrados iguais e vazam padrões (figura 1). Implementado apenas para demonstração.
- **CBC:** $C_i = E_K(P_i \oplus C_{i-1})$, com C_0 = IV aleatório e padding **PKCS#7**.
- **CTR:** $C_i = P_i \oplus E_K(\text{contador} + i)$. Funciona como cifra de fluxo, sem padding, e permite paralelizar.



Figura 1 – Saída real de `demo_ecb_vs_cbc.py` : um desenho de 288 blocos cifrado com AES. No modo ECB o cadeado continua visível (só 2 blocos cifrados distintos); no CBC, os 288 blocos são distintos.

Formato do arquivo cifrado do SecureDocs (`arquivo.py`):

```
| "SDOC" (4 B) | versão (1 B) | salt (16 B) | IV (16 B) | texto cifrado AES-256-CBC (n B) | HMAC-SHA256 (32 B) |
```

- **Chave derivada da senha** com PBKDF2-HMAC-SHA256 (200.000 iterações e salt aleatório): 32 bytes para o AES-256 e 32 bytes para o HMAC. O salt faz a mesma senha gerar chaves diferentes em cada arquivo, e as iterações deixam lentos os ataques de dicionário.
- **IV aleatório** e novo a cada arquivo: o mesmo documento cifrado duas vezes gera arquivos diferentes.
- **Encrypt-then-MAC:** o HMAC cobre cabeçalho e cifrado e é verificado *antes* da decifragem, com comparação em tempo constante. Isso responde ao incidente do **contrato adulterado**: qualquer alteração faz o arquivo ser recusado.

7. Experimento: Arquivo → AES → Cifrado → AES → Arquivo

Script `exemplos/demo_aes_arquivo.py`, executado sobre um contrato fictício de 681 bytes. Saída real (trechos):

```
[1] Arquivo original : contrato_confidencial.txt (681 bytes)
    SHA-256          : 2a2a67bd95b088a2a1e2280e9cfa3269bb6dc2f5620c708dce8dc0cb95bfd8a0
    Entropia         : 5.12 bits/byte
   0000  43 4f 4e 54 52 41 54 4f 20 44 45 20 50 52 45 53  CONTRATO DE PRES

[2] AES-256-CBC + HMAC-SHA256 -> contrato_confidencial.txt.sdoc (757 bytes)
    Entropia         : 7.68 bits/byte
    Contém 'R$ 1.250.000'? NÃO
   0000  53 44 4f 43 01 3b 69 74 d0 75 59 91 de 3e eb 0c  SDOC.;it.uY..>..
   0010  21 32 34 a5 9e 11 0e 2c 14 da 6c 7e ca 50 46 ef  !24....,..l~.PF.

[3] AES-256 (decifrar) -> contrato_recuperado.txt
    SHA-256          : 2a2a67bd95b088a2a1e2280e9cfa3269bb6dc2f5620c708dce8dc0cb95bfd8a0
    Idêntico ao original? SIM

[4] Ataque: invasor altera 1 bit do arquivo cifrado (contrato adulterado)
    Documento RECUSADO -> Falha de integridade: arquivo adulterado ou senha incorreta.

[5] Ataque: invasor tenta abrir o arquivo roubado com outra senha
    Documento RECUSADO -> Falha de integridade: arquivo adulterado ou senha incorreta.
```

Conclusões: o arquivo recuperado tem exatamente o mesmo SHA-256 do original, então o ciclo é perfeito. O cifrado não contém nenhum trecho legível, e a entropia sobe de 5,12 para 7,68 bits/byte (perto do máximo de 8 para um arquivo pequeno). Os 76 bytes extras são o cabeçalho (37 B), o padding (7 B) e o HMAC (32 B). Um arquivo roubado só serve ao invasor se ele tiver a chave.

8. Testes e ataques realizados

8.1 Validação por vetores oficiais

18 testes automatizados (`python -m unittest discover -s tests -v`), **todos aprovados**:

Algoritmo	Fonte do vetor	Chave	Texto claro	Cifrado esperado = obtido
DES	Exemplo clássico FIPS 46	`133457799BBCDFF1`	`0123456789ABCDEF`	`85E813540F0AB405` ✓
DES	NIST SP 800-17	`0101010101010101`	`8000000000000000`	`95F8A5E5DD31D900` ✓
RC5-32/12/16	Rivest (1994)	`00…00`	`0000000000000000`	`21A5DBEE154B8F6D` ✓
RC5-32/12/16	Rivest (1994)	`915F4619…A9CE91`	`21A5DBEE154B8F6D`	`F7C013AC5B2B8952` ✓
AES-128	FIPS 197 C.1	`000102…0E0F`	`00112233…EEFF`	`69C4E0D86A7B0430…` ✓
AES-192	FIPS 197 C.2	`000102…1617`	`00112233…EEFF`	`DDA97CA4864CDFE0…` ✓
AES-256	FIPS 197 C.3	`000102…1E1F`	`00112233…EEFF`	`8EA2B7CA516745BF…` ✓
AES-CBC / CTR	NIST SP 800-38A	`2B7E1516…4F3C`	`6BC1BEE2…172A`	`7649ABAC…` / `874D6191…` ✓

8.2 Ataque de força bruta ao DES (prova de conceito)

O script `ataque_forca_bruta_des.py` simula o invasor com um par conhecido (todo contrato começa com "CONTRATO"). Com 14 bits desconhecidos, a chave foi encontrada após **12.088 tentativas em 3,2 s** (≈ 3.800 chaves/s em Python puro). Extrapolando o tempo médio (metade do espaço de chaves):

Atacante	Velocidade	DES (2^{56})	3DES (2^{112})	AES-128	AES-256
Notebook, Python puro	3,8 mil/s	$3{,}0 \cdot 10^{5}$ anos	$2{,}2 \cdot 10^{22}$ anos	$1{,}4 \cdot 10^{27}$ anos	$4{,}8 \cdot 10^{65}$ anos
Deep Crack (EFF, 1998)	$9 \cdot 10^{10}$/s	**4,6 dias**	$9{,}1 \cdot 10^{14}$ anos	$6{,}0 \cdot 10^{19}$ anos	$2{,}0 \cdot 10^{58}$ anos
Cluster de GPUs atual	10^{12}/s	**10 horas**	$8{,}2 \cdot 10^{13}$ anos	$5{,}4 \cdot 10^{18}$ anos	$1{,}8 \cdot 10^{57}$ anos

Para comparação, a idade do universo é de cerca de $1{,}4 \cdot 10^{10}$ anos.

8.3 Desempenho (Python puro, modo CBC, 32 KB)

Algoritmo	Chave	Bloco	Rodadas	Cifrar (KB/s)	Decifrar (KB/s)
DES	56	64	16	47,1	39,8
3DES	112	64	48	15,1	15,7
RC5-32/12/16	128	64	12	705,1	784,6
AES-128	128	128	10	316,1	323,0
AES-256	256	128	14	266,7	258,4

O RC5 é o mais rápido *em software puro* (só soma, XOR e rotação). O DES é lento porque permuta bits um a um, uma operação pensada para hardware. Em processadores atuais, porém, as instruções **AES-NI** cifram AES a vários GB/s, o que o torna o mais rápido na prática.

9. Comparativo DES × RC5 × AES

Critério	DES	RC5	AES
Criação / padrão	IBM, 1977 – FIPS 46	Rivest, 1994 – RFC 2040	Daemen e Rijmen, 2001 – FIPS 197
Estrutura	Feistel, 16 rodadas	Tipo Feistel, r rodadas	SPN, 10/12/14 rodadas
Bloco	64 bits	32/64/128 bits (2w)	128 bits
Chave	**56 bits**	0–2040 bits (usual 128)	**128/192/256 bits**
Não linearidade	8 S-boxes fixas	Rotações dependentes dos dados	S-box algébrica em GF(2^8)
Melhor ataque conhecido	**Força bruta prática (horas)**	Diferencial 2^{44} (12 rodadas)	**Biclique $2^{126,1}$ (teórico)**
Resistência quântica (Grover)	2^{28}	2^{64} (chave 128)	2^{128} **(AES-256)**
Aceleração em hardware	Obsoleta	Não	**AES-NI, ARMv8-CE**
Situação atual	**Retirado (2005)**	Pouco usado, não padronizado	**Padrão mundial (TLS, VPN, discos)**

10. Justificativa: qual algoritmo usar no SecureDocs

> **Decisão da equipe: AES com chave de 256 bits.**

1. **Segurança comprovada:** é padrão do NIST, ISO/IEC 18033-3 e da maioria das regulamentações. Resistiu a 25 anos de criptoanálise pública sem ataque prático.
2. **Chave e bloco adequados:** 256 bits tornam a força bruta fisicamente impossível, e o bloco de 128 bits elimina os ataques de aniversário (Sweet32) que afetam DES, 3DES e RC5-32.

3. **Ameaças futuras:** mesmo com o algoritmo de Grover, o AES-256 mantém 128 bits de segurança. Os contratos e documentos estratégicos da TechSecure podem precisar ficar sigilosos por décadas, e um invasor pode guardar os arquivos roubados hoje para tentar quebrá-los depois (*harvest now, decrypt later*).
4. **Desempenho e ecossistema:** tem aceleração em hardware em praticamente todo processador e bibliotecas maduras e auditadas.
5. **Por que não os outros:** o DES cai em horas. O 3DES é lento, tem bloco de 64 bits e já foi descontinuado. O RC5 com 12 rodadas tem ataque diferencial conhecido, não é padronizado e não tem suporte de hardware.

Como usar: o AES deve rodar em um modo seguro, com IV ou nonce único e autenticação. Na prova de conceito usamos **AES-256-CBC + HMAC-SHA256** (Encrypt-then-MAC). Em produção, a recomendação é o **AES-256-GCM** (cifragem autenticada, AEAD) de uma biblioteca auditada, como `cryptography` ou OpenSSL. Ela dá o mesmo resultado com mais desempenho e proteção contra ataques de canal lateral.

11. Nova descoberta: como entregar a chave secreta ao destinatário sem que um invasor também consiga obtê-la?

Este é o **problema da distribuição de chaves**, o calcanhar de Aquiles da criptografia simétrica. O AES é forte, mas a chave precisa estar com o remetente *e* com o destinatário. Se ela for enviada pela mesma Internet que o invasor monitora, ele também a obtém, e toda a cifragem perde o sentido. Além disso, com n usuários seriam necessárias $n(n-1)/2$ chaves: para os 1.200 usuários do SecureDocs, quase 720 mil chaves.

11.1 Alternativas estudadas

Solução	Como funciona	Limitação
Canal alternativo (*out-of-band*)	Entregar a chave pessoalmente, por telefone, mensageiro ou pendrive.	Não escala, é lento e vulnerável a erro humano.
Centro de distribuição de chaves (KDC / Kerberos)	Um servidor confiável compartilha uma chave com cada usuário e gera chaves de sessão sob demanda.	Ponto único de falha e de confiança. Exige que todos confiem no servidor.
Acordo de chaves Diffie-Hellman (DH / ECDH, X25519)	Cada parte envia apenas um valor *público*, e ambas calculam o mesmo segredo. Quem só observa a troca não consegue calculá-lo (problema do logaritmo discreto).	Sozinho não autentica, então é vulnerável a *man-in-the-middle*. Precisa de certificados ou assinaturas.
Criptografia híbrida (envelope digital)	O documento é cifrado com uma chave AES aleatória, e *essa chave* é cifrada com a **chave pública** do destinatário (RSA-OAEP, ECIES ou ML-KEM).	Exige uma infraestrutura de chaves públicas (PKI) para confirmar de quem é cada chave pública.

11.2 Resposta da equipe

> **A chave secreta não deve viajar em claro.** Usa-se **criptografia assimétrica** para transportar ou combinar a chave simétrica, e o AES para cifrar os dados. Esse esquema é o **envelope digital**, e é como funcionam TLS 1.3, PGP, S/MIME e os serviços de nuvem (KMS).

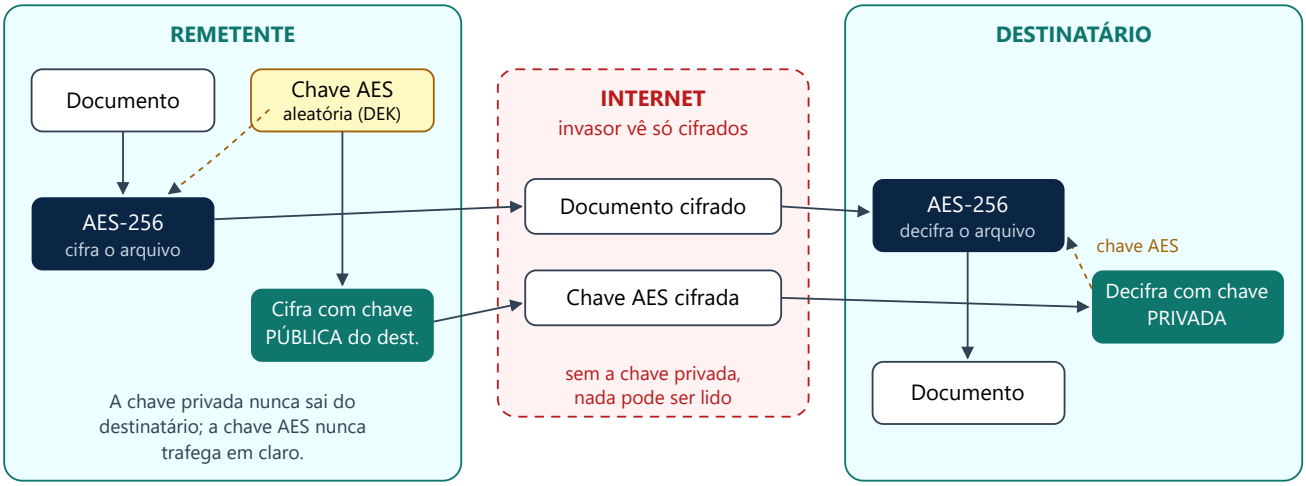


Figura 2 – Envelope digital: o AES protege os dados e a criptografia assimétrica protege a chave do AES.

Proposta de arquitetura para o SecureDocs:

1. Cada usuário tem um par de chaves (pública/privada). As chaves públicas são publicadas em **certificados digitais** emitidos pela autoridade certificadora da TechSecure (PKI), o que impede o *man-in-the-middle*.
2. Para cada documento é gerada uma **chave AES-256 aleatória e única** (DEK). O documento é cifrado com AES-256-GCM.
3. A DEK é cifrada ("embrulhada") com a chave pública de cada destinatário autorizado e guardada junto ao documento. Assim, um único arquivo pode ser compartilhado com vários usuários sem que ninguém precise conhecer a chave de outro.
4. A chave privada fica no dispositivo do usuário ou em um módulo de hardware (HSM/KMS) e nunca é enviada.
5. Toda comunicação passa por **TLS 1.3**, em que as partes fazem ECDHE, uma variante efêmera de Diffie-Hellman: mesmo que uma chave vaze no futuro, as sessões passadas continuam protegidas (*forward secrecy*).
6. **Ameaças futuras:** como RSA e ECDH serão quebrados por um computador quântico de grande porte, recomenda-se adotar o modo híbrido **X25519 + ML-KEM-768** (FIPS 203, 2024), que já é usado no TLS dos principais navegadores.

Os próximos conteúdos da disciplina (criptografia assimétrica, funções hash e assinaturas digitais) serão as próximas missões. Eles completam a arquitetura, garantindo também **autenticidade** e **não repúdio**.

12. A biblioteca entregue

```
biblioteca/
├── securedocs_crypto/
│   ├── __init__.py      API pública
│   ├── __main__.py      linha de comando: python -m securedocs_crypto cifrar|decifrar
│   ├── des.py           DES + 3DES
│   ├── rc5.py           RC5-w/r/b
│   ├── aes.py           AES-128/192/256 (S-box calculada)
│   ├── modos.py         ECB, CBC, CTR, PKCS#7
│   └── arquivo.py       AES-256-CBC + HMAC-SHA256 + PBKDF2 (formato .sdoc)
├── tests/test_vetores.py          18 testes com vetores oficiais
└── exemplos/
    ├── demo_aes_arquivo.py        experimento da missão
    ├── ataque_forca_bruta_des.py  força bruta + estimativas
    ├── comparativo_desempenho.py  vazão de cada algoritmo
    ├── demo_ecb_vs_cbc.py         vazamento de padrões no ECB
    └── saida/                     resultados obtidos
```

Python puro, sem dependências externas. Exemplo de uso:

```
from securedocs_crypto import AES, cbc_encrypt, cbc_decrypt
cifra = AES(chave_de_32_bytes)                # AES-256
iv, cifrado = cbc_encrypt(cifra, b"relatorio financeiro")
original = cbc_decrypt(cifra, iv, cifrado)
```

> Código **didático**: não é protegido contra ataques de canal lateral (tempo, cache) e é lento. Em produção, devem ser usadas bibliotecas auditadas.

13. Referências

- NIST. *FIPS PUB 46-3 – Data Encryption Standard (DES)*. 1999.
- NIST. *FIPS PUB 197 – Advanced Encryption Standard (AES)*. 2001 (atualizado em 2023).
- NIST. *SP 800-38A – Recommendation for Block Cipher Modes of Operation*. 2001.
- NIST. *SP 800-131A Rev. 2 – Transitioning the Use of Cryptographic Algorithms and Key Lengths*. 2019.
- NIST. *FIPS 203 – Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM)*. 2024.
- RIVEST, R. *The RC5 Encryption Algorithm*. Fast Software Encryption, LNCS 1008, 1995.
- BALDWIN, R.; RIVEST, R. *RFC 2040 – The RC5, RC5-CBC, RC5-CBC-Pad, and RC5-CTS Algorithms*. 1996.
- BIRYUKOV, A.; KUSHILEVITZ, E. *Improved Cryptanalysis of RC5*. EUROCRYPT 1998.
- BOGDANOV, A.; KHOVRATOVICH, D.; RECHBERGER, C. *Biclique Cryptanalysis of the Full AES*. ASIACRYPT 2011.
- ELECTRONIC FRONTIER FOUNDATION. *Cracking DES: Secrets of Encryption Research, Wiretap Politics & Chip Design*. 1998.
- BHARGAVAN, K.; LEURENT, G. *On the Practical (In-)Security of 64-bit Block Ciphers (Sweet32)*. ACM CCS 2016.
- STALLINGS, W. *Criptografia e Segurança de Redes: Princípios e Práticas*. 6. ed. Pearson, 2015.
- DIFFIE, W.; HELLMAN, M. *New Directions in Cryptography*. IEEE Trans. Inf. Theory, 1976.